



SKAO Regional Centre **United Kingdom**

Performance comparison of the visibility simulators

UKSRC MEMO

Document ID	UKSRC-MEMO-0002
Revision	0.2
Author(s)	Vladislav Stolyarov, Peter Sims, Quentin Gueuning, Ed Polehampton, Danny Lloyd, Tianyue Chen
Contact	Vladislav Stolyarov (vs237@cam.ac.uk)
Release date	30 June 2026
Document classification	Unrestricted
Status	Draft
Tags	validation, EoR, benchmarking, jira:SE-324, jira:SE-388

UKSRC Memo Disclaimer

The UKSRC memos are intended to support the rapid recording of technical work, investigations, design notes, and open questions. A memo may reflect the views of its authors at the time of writing and should not automatically be taken as formal UKSRC policy, project baseline, or final design guidance.

Executive summary

This memo presents benchmarking results comparing the performance of the visibility simulators OSKAR¹, pyuvsim², fftvis³, and matvis⁴. Each simulator has its own pros and cons; shorter computation times typically come at the cost of reduced astrometric accuracy.

Document history

Revision	Date of issue	Comments
0.1	30 June 2026	Initial draft

¹<https://github.com/OxfordSKA/OSKAR>

²<https://github.com/RadioAstronomySoftwareGroup/pyuvsim>

³<https://github.com/tyler-a-cox/fftvis>

⁴<https://github.com/HERA-Team/matvis>

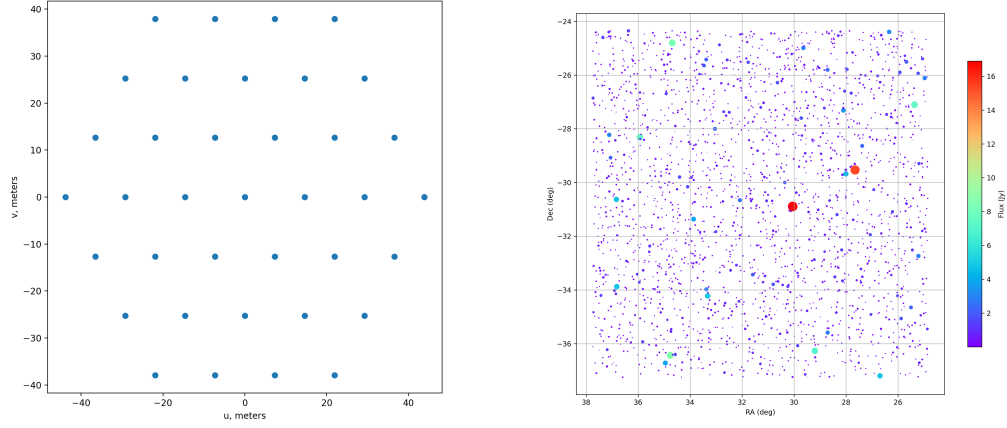


Figure 1: Left: HERA instrument antenna positions; right: GLEAM field map used in simulations.

Table of Contents

1	Introduction	3
2	Simulator tests with different computer configurations	4
2.1	Dell Pro Max Tower T2 Desktop	4
2.2	SKA SLURM Cluster Performance	5
2.3	CSD3 Icelake cluster performance	6
2.3.1	CSD3 Performance with respect to the number of MPI threads N_{mpi}	7
3	Conclusions and Recommendations	7

1 Introduction

Visibility simulators play a crucial role in the design of radio interferometric projects and the validation of data processing pipelines. Several visibility simulation software packages are available on GitHub or GitLab, each developed for different projects. Notable examples include `OSKAR` [Dulwich et al., 2009], `pyuvsim` [Lanman et al., 2019], `fftvis` [Cox et al., 2025], and `matvis` [Kittiwisit et al., 2025]. A performance comparison of these simulators was conducted using the HERA telescope model and a subset of the GLEAM catalogue containing 2243 point sources. The antenna positions and the GLEAM field map are shown in Fig. 1.

The simulation parameters are listed in Table 2.

Telescope model	HERA, 37 antennas, $N_{\text{base}} = 666$
N_{times}	360
t_{int} , sec	11
Starting frequency ν_{start} , MHz	158.3
Channel width, $\Delta\nu$, kHz	23761.8
N_{freqs}	1,2,4,8,16,32,64,128,256,512,1024,2048,4096
N_{sources} to simulate	2243

Table 2: Simulation parameters

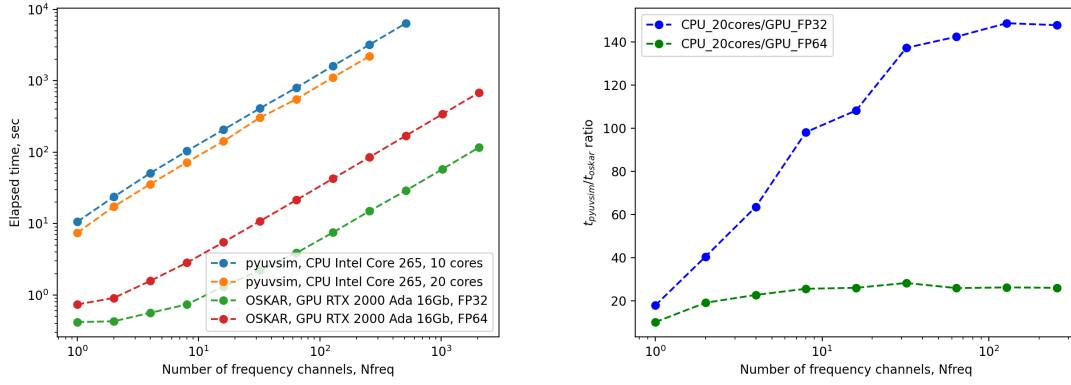


Figure 2: Left: Elapsed time vs N_{freq} for `pyuvsim` using 10 and 20 cores and `OSKAR` for FP32 and FP64; Right: elapsed time ratio between `OSKAR` and `pyuvsim` 20 cores simulations.

2 Simulator tests with different computer configurations

The above-mentioned simulators were tested on different computers, including desktops and HPC clusters. Below are the results for some of the used systems.

2.1 Dell Pro Max Tower T2 Desktop

This desktop features a 20-core Intel Core Ultra 7 265K CPU (8 P-cores + 12 E-cores), an NVIDIA RTX 2000 Ada Generation GPU with 16 GB of memory, and 64 GB of installed RAM.

Fig. 2 compares the simulation elapsed time for `pyuvsim` and `OSKAR`. It shows how the elapsed time varies with the number of frequency channels, N_{freq} , as well as the ratio of elapsed times between the two simulators. Overall, `OSKAR` is up to 140 times faster when simulations are run in single precision, provided the simulation scale is sufficient to fully utilize the GPU.

All four simulators are compared in Fig. 3, revealing that `fftvis` is the fastest among them when operating in double precision.

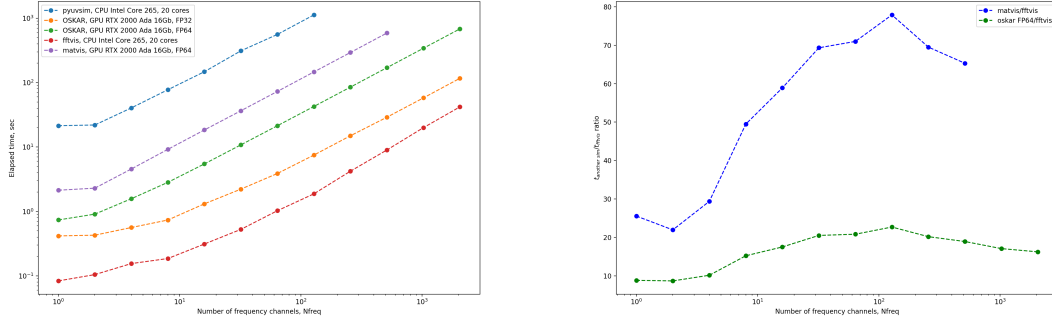


Figure 3: Left: Elapsed time vs N_{freq} for all four simulators; Right: elapsed time ratio $\text{OSKAR}/\text{fftvis}$ and $\text{matvis}/\text{fftvis}$ 20 cores simulations.

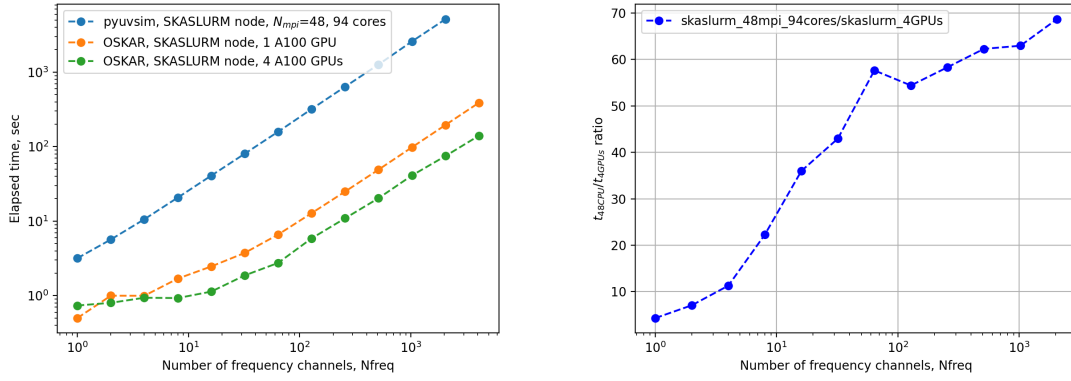


Figure 4: Left: Elapsed time vs N_{freq} for `pyuvsim` and `OSKAR` using 1 GPU and 4 GPUs on the compute node; Right: elapsed time ratio between `pyuvsim` and `OSKAR` using all 4 GPUs for simulations.

2.2 SKA SLURM Cluster Performance

The SKA SLURM cluster consists of CPU and GPU nodes. A CPU node features $2 \times$ Intel Sapphire Rapids CPUs, providing up to 94 hardware cores, 188 hyper-threading (HT) threads, and 1862.2 GB of RAM. A GPU node is equipped with 4 A100-SXM4-80GB GPUs, an AMD 120-core CPU, and 897.1 GB of RAM.

Fig. 4 compares the performance of `pyuvsim` and `OSKAR` on a single node of the SKA SLURM cluster.

Poor MPI scaling was observed for `pyuvsim` when using more than one node, presumably due to inter-node communication volume creating a bottleneck. On a single node, the optimal number of MPI threads is approximately half of the available CPU cores, $N_{\text{mpi}} = N_{\text{cores}}/2$. This bottleneck, currently under investigation, is illustrated in Fig. 5. The simulation setup remains fixed with $N_{\text{freq}} = 256$, while N_{mpi} varies from 8 on a single node to 256 across three nodes. The same effect was observed on the CSD3 cluster.

The `fftvis` simulator was also tested on a CPU node of the SKA SLURM cluster, demonstrating

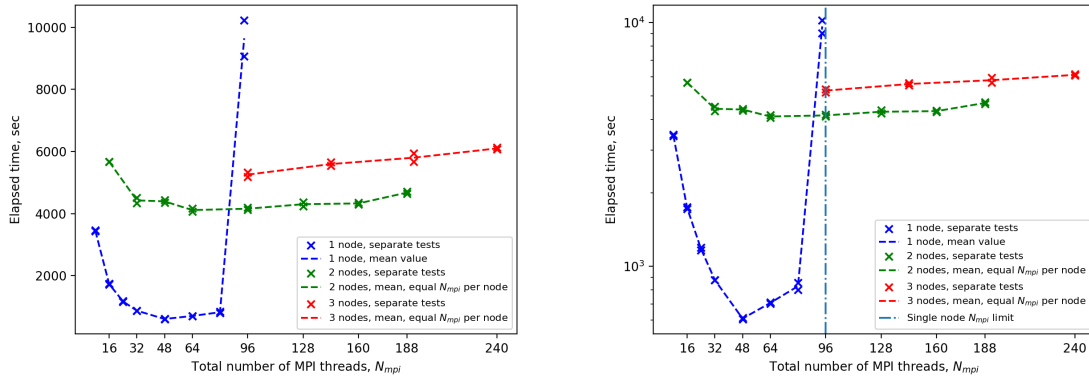


Figure 5: Elapsed time vs. N_{mpi} threads for `pyuvsim` on 1, 2, and 3 compute nodes: right – linear plot; left – semi-log plot. The optimum for a single node is achieved at $N_{\text{mpi}} = 48$; performance degrades when using more than one node.

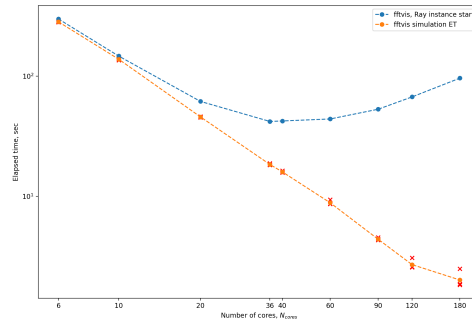


Figure 6: Elapsed time vs N_{threads} threads for a single node for `fftvis`. The time required to start Ray cluster is accounted separately.

significantly better scaling with respect to N_{threads} . However, note that `fftvis` relies on the Ray⁵ infrastructure for distributed workload management, and initial startup incurs extra time for configuring the Ray cluster. The scaling with respect to N_{threads} is shown in Fig. 6.

2.3 CSD3 Icelake cluster performance

A series of tests was conducted on the CSD3 cluster⁶ using a single Icelake node equipped with two Intel Xeon Platinum 8368Q CPUs (2×38 cores per node) and 256 GB of RAM per node. The results are shown in Figure 7, alongside comparisons with the Intel Core Ultra 7 265 (20 cores). In this configuration, the available RAM is sufficient to run simulations with $N_{\text{freq}} = 1024$ frequency channels. However, the elapsed time exceeds two hours in this case. The primary advantage of the CSD3 node lies in its large memory capacity, while the execution time improves only marginally with respect to the number of cores,

⁵<https://www.ray.io/>

⁶<https://docs.hpc.cam.ac.uk/hpc/user-guide/icelake.html>

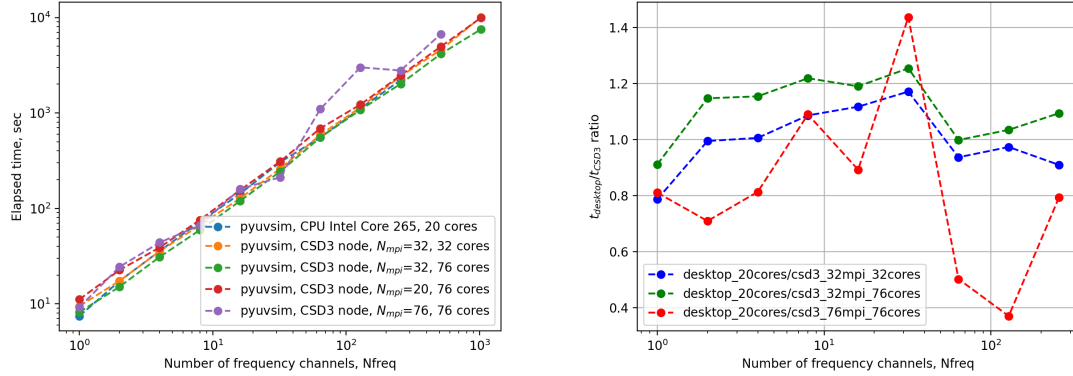


Figure 7: Left: Elapsed time vs. N_{freq} for `pyuvsim` on a single Icelake node, with $N_{\text{cores}} = 32$ and 76 allocated via the SLURM submission script and $N_{\text{mpi}} = 20, 32$, and 76, compared with a desktop Intel Core Ultra 7 265K (20 cores). Right: Ratio of elapsed time between the desktop 20-core CPU and the CSD3 simulations. The improvement is quite marginal at this simulation scale.

likely due to parallel bottlenecks inherent in the algorithm.

It is important to note that the scatter in the elapsed time is large, likely due to uneven cluster load, as the scheduler must allocate resources to many users simultaneously. An example of this elapsed time scatter for the `pyuvsim` tests ($N_{\text{mpi}} = 32$, $N_{\text{cores}} = 32$, with half of the node allocated to the job) is shown in Fig. 8.

2.3.1 CSD3 Performance with respect to the number of MPI threads N_{mpi}

To characterize the performance of `pyuvsim` as a function of the number of MPI threads N_{mpi} , additional tests were conducted. The simulation setup was identical to that described above ($N_{\text{sources}} = 2243$, $N_{\text{times}} = 360$), with a fixed number of frequency channels $N_{\text{freq}} = 256$, while varying N_{mpi} . The number of CPU cores allocated per node was held constant at $N_{\text{cores}} = 76$, meaning that all nodes were dedicated solely to this test. Simulations were run on either 1 or 2 compute nodes, and the measured times are shown in Fig. 9. It is clear that performance degrades when using two nodes compared to single-node tests, where the optimum is reached when $N_{\text{mpi}} \simeq N_{\text{cores}}/2$. Also, it is worth noting that the same bottleneck regarding N_{mpi} is observed on the SKA SLURM cluster (see Fig. 5).

3 Conclusions and Recommendations

Summarise the main outcome, recommendation, or decision.

References

Tyler A. Cox, Steven G. Murray, Aaron R. Parsons, Joshua S. Dillon, Kartik Mandar, Zachary E. Martinot, Robert Pascua, Piyanat Kittiwisit, and James E. Aguirre. `fftvis`: a non-uniform Fast Fourier Transform based interferometric visibility simulator. *RAS Techniques and Instruments*, 4:rzaf056, January 2025. doi: 10.1093/rasti/rzaf056.

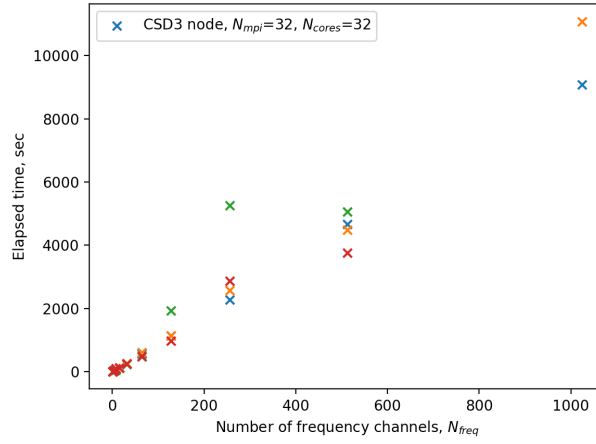


Figure 8: An illustration of the elapsed time scatter for the same simulation scale on the CSD3 Icelake node.

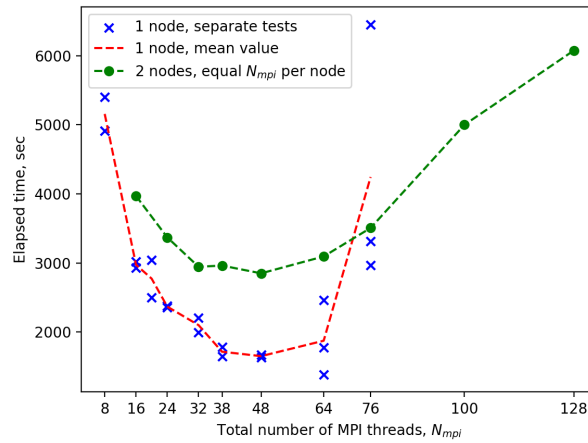


Figure 9: Elapsed time vs. N_{mpi} threads for `pyuvsim` on 1 and 2 Icelake compute nodes. The optimum for a single node is achieved at $N_{mpi} = 32$ (approximately $N_{cores}/2$); performance degrades when using more than one node.

F. Dulwich, B. J. Mort, S. Salvini, K. Zarb Adami, and M. E. Jones. OSKAR: Simulating Digital Beamforming for the SKA Aperture Array. In *Wide Field Astronomy & Technology for the Square Kilometre Array*, page 31, January 2009. doi: 10.22323/1.132.0031.

Piyanat Kittiwisit, Steven G. Murray, Hugh Garsden, Philip Bull, Michael J. Wilensky, Christopher Cain, Aaron R. Parsons, Jackson Sipple, Tyrone Adams, James E. Aguirre, Rushelle Baartman, Adam P. Beardsley, Lindsay M. Berkhout, Gianni Bernardi, Tashalee S. Billings, Judd D. Bowman, Richard F. Bradley, Jacob Burba, Steven Carey, Chris L. Carilli, Kai-Feng Chen, Samir Choudhuri, Tyler Cox, David R. DeBoer, Eloy de Lera Acedo, Matt Dexter, Joshua S. Dillon, Nico Eksteen, John Ely, Aaron Ewall-Wice, Nicolas Fagnoni, Steven R. Furlanetto, Kingsley Gale-Sides, Bharat Kumar Gehlot, Brian Glendenning, Adelie Gorce, Deepthi Gorthi, Bradley Greig, Jasper Grobbelaar, Ziyaad Halday, Bryna J. Hazelton, Jacqueline N. Hewitt, Jack Hickish, Daniel C. Jacobs, Alec Josaitis, Nicholas S. Kern, Joshua Kerrigan, Honggeun Kim, Matthew Kolopanis, Adam Lanman, Paul La Plante, Adrian Liu, Yin-Zhe Ma, David H. E. MacMahon, Lourence Malan, Cresshim Malgas, Keith Malgas, Bradley Marero, Zachary E. Martinot, Lisa McBride, Andrei Mesinger, Mathakane Molewa, Miguel F. Morales, Tshegofalang Mosiane, Chuneeta Devi Nunhokee, Hans Nuwegeld, Robert Pascua, Yuxiang Qin, Eleanor Rath, Nima Razavi-Ghods, James Robnett, Mario G. Santos, Peter Sims, Saurabh Singh, Dara Storer, Hilton Swarts, Jianrong Tan, Nithyanandan Thyagarajan, Pieter van Wyngaarden, Zhilei Xu, and Haoxuan Zheng. matvis: a matrix-based visibility simulator for fast forward modelling of many-element 21 cm arrays. *RAS Techniques and Instruments*, 4:rza001, January 2025. doi: 10.1093/rasti/rza001.

Adam Lanman, Bryna Hazelton, Daniel Jacobs, Matthew Kolopanis, Jonathan Pober, James Aguirre, and Nithyanandan Thyagarajan. pyuvsim: A comprehensive simulation package for radio interferometers in python. *The Journal of Open Source Software*, 4(37):1234, May 2019. doi: 10.21105/joss.01234.